



Date: __ / __ / __

Experiment 8

8. a. You're tasked with implementing Kruskal's algorithm in the C programming language to find the Minimum Cost Spanning Tree (MCST) for a given weighted graph. The algorithm should efficiently determine the subset of edges that connect all vertices of the graph with the minimum total weight. Design two functions called find and union. The find function is used to determine the representatives of the sets containing the source and destination vertices of the edge. The Union function is used to merge two disjoint sets into a single set by connecting their representatives.

For more information, students are requested to read chapter 9.2 from the textbook 1. (Levitin, A. (2011). Introduction to the Design and Analysis of Algorithms. Always learning.)

Type of Lab Activity: Exercise

Aim	Material	Methods	Answer
Given	Given	Given as Algorithm	Open

ALGORITHM *Kruskal(G)*

```
//Kruskal's algorithm for constructing a minimum spanning tree
//Input: A weighted connected graph  $G = \langle V, E \rangle$ 
//Output:  $E_T$ , the set of edges composing a minimum spanning tree of  $G$ 
sort  $E$  in nondecreasing order of the edge weights  $w(e_{i_1}) \leq \dots \leq w(e_{i_{|E|}})$ 
 $E_T \leftarrow \emptyset$ ;  $ecounter \leftarrow 0$  //initialize the set of tree edges and its size
 $k \leftarrow 0$  //initialize the number of processed edges
while  $ecounter < |V| - 1$  do
     $k \leftarrow k + 1$ 
    if  $E_T \cup \{e_{i_k}\}$  is acyclic
         $E_T \leftarrow E_T \cup \{e_{i_k}\}$ ;  $ecounter \leftarrow ecounter + 1$ 
return  $E_T$ 
```

Program

(Only for faculty reference, should not be shared to students)

```
#include <stdio.h>
#include <stdlib.h>
```

```
// Structure to represent an edge in the graph
struct Edge {
    int src, dest, weight;
};
```



```
// Structure to represent a subset for union-find
struct Subset {
    int parent;
    int rank;
};

// Function prototypes
int find(struct Subset subsets[], int i);
void Union(struct Subset subsets[], int x, int y);
int compareEdges(const void* a, const void* b);
void KruskalMST(struct Edge edges[], int V, int E);

int main() {
    int V, E;
    printf("Enter the number of vertices and edges: ");
    scanf("%d %d", &V, &E);

    struct Edge* edges = (struct Edge*)malloc(E * sizeof(struct Edge));
    printf("Enter source, destination, and weight for each edge:\n");
    for (int i = 0; i < E; ++i)
        scanf("%d %d %d", &edges[i].src, &edges[i].dest, &edges[i].weight);

    KruskalMST(edges, V, E);

    free(edges);
    return 0;
}

// Find set of an element i (uses path compression technique)
int find(struct Subset subsets[], int i) {
    if (subsets[i].parent != i)
        subsets[i].parent = find(subsets, subsets[i].parent);
    return subsets[i].parent;
}

// Union of two sets of x and y (uses union by rank)
void Union(struct Subset subsets[], int x, int y) {
    int xroot = find(subsets, x);
    int yroot = find(subsets, y);

    if (subsets[xroot].rank < subsets[yroot].rank)
        subsets[xroot].parent = yroot;
    else if (subsets[xroot].rank > subsets[yroot].rank)
        subsets[yroot].parent = xroot;
    else {
        subsets[yroot].parent = xroot;
        subsets[xroot].rank++;
    }
}
```



```
// Comparator function for sorting edges based on weight
int compareEdges(const void* a, const void* b) {
    struct Edge* edge1 = (struct Edge*)a;
    struct Edge* edge2 = (struct Edge*)b;
    return edge1->weight - edge2->weight;
}

// Kruskal's algorithm for finding Minimum Spanning Tree
void KruskalMST(struct Edge edges[], int V, int E) {
    qsort(edges, E, sizeof(edges[0]), compareEdges);

    struct Subset* subsets = (struct Subset*)malloc(V * sizeof(struct Subset));

    for (int v = 0; v < V; ++v) {
        subsets[v].parent = v;
        subsets[v].rank = 0;
    }

    struct Edge* result = (struct Edge*)malloc((V - 1) * sizeof(struct Edge));
    int e = 0; // Index for result[]

    int i = 0; // Index for sorted edges[]
    while (e < V - 1 && i < E) {
        struct Edge next_edge = edges[i++];

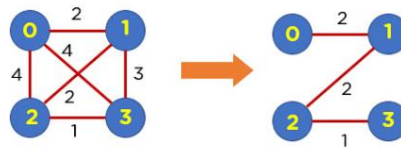
        int x = find(subsets, next_edge.src);
        int y = find(subsets, next_edge.dest);

        if (x != y) {
            result[e++] = next_edge;
            Union(subsets, x, y);
        }
    }

    printf("Edges in the Minimum Spanning Tree:\n");
    for (int j = 0; j < e; ++j)
        printf("%d -- %d == %d\n", result[j].src, result[j].dest, result[j].weight);

    free(subsets);
    free(result);
}
```

Sample Output



Enter the number of vertices and edges: 4 6

Enter source, destination, and weight for each edge:

0 1 2

1 3 3

2 3 1

0 2 4

0 3 4

2 1 2

Edges in the Minimum Spanning Tree:

2 -- 3 == 1

0 -- 1 == 2

2 -- 1 == 2